

```

#include<iostream>
using namespace std;
//The function will accept the arguments via the parameters,
//perform the necessary processing,
//and print the final result.
void addition(int no1,int no2)//stored in n1-->no1  n2--->no2
{
    //no1,no2:1. Local copy of data kept only within the function when the function is over
    cout<<"\naddress of no1 and no2 "<<&no1<<",<<&no2;
    cout<<"\n"<<no1<<+" "<<no2<<+"="<<(no1+no2);
}

```

```

int main()
{
//The job of reading data and passing it to the function is done by the main script. The actual
processing of the data and the result printing are done by the function.
    int n1,n2;
    cout<<"Enter two numbers. \n";
    cin>>n1>>n2;
    cout<<"\naddress of n1 and n2 "<<&n1<<",<<&n2;
    addition(n1,n2);//pass by value
    return 0;
}

```

//Create a function called `area` which will accept a radius and print the area of a circle.

```

#include<iostream>
using namespace std;
    //If a function returns data either
    //we print it directly using `cout` or
    //we store it in a variable and then print it.
float area(float r)
{
    return(3.14*r*r);
    //When we use `return`, the type of data it is going to return
    //would become the return type of the function.
}

int main()
{
    float radius;
    cout<<"Enter radius:";
    cin>>radius;
    float a=area(radius);
    cout<<"\nArea is:"<<a;

    cout<<"\nArea is:"<<area(radius);//this is better
    return 0;
}

```

```

#include<iostream>
using namespace std;
int min(int no1,int no2)
{
    if(no1<no2)
        return no1;
    else
        return no2;
}

int main()
{
    int no1,no2,no3,no4;
    //without modifying fucntion find min of 4 and print it
    //assume no duplicates
    return 0;
}

```

```

#include <iostream>
using namespace std;

```

```

// 1. Pass by Value
void swapByValue(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}

```

```

// 2. Pass by Reference
void swapByReference(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

```

```

// 3. Pass by Address / Pointer
void swapByAddress(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

```

int main()
{
    int a, b;

```

```

cout << "Enter two numbers: ";
cin >> a >> b;

cout << "\nOriginal values:";
cout << "\na = " << a << ", b = " << b << endl;

// -----
// PASS BY VALUE
// -----
int x = a;
int y = b;

swapByValue(x, y);

cout << "\nAfter Pass by Value:";
cout << "\nx = " << x << ", y = " << y << endl;

// -----
// PASS BY REFERENCE
// -----
int p = a;
int q = b;

swapByReference(p, q);

cout << "\nAfter Pass by Reference:";
cout << "\np = " << p << ", q = " << q << endl;

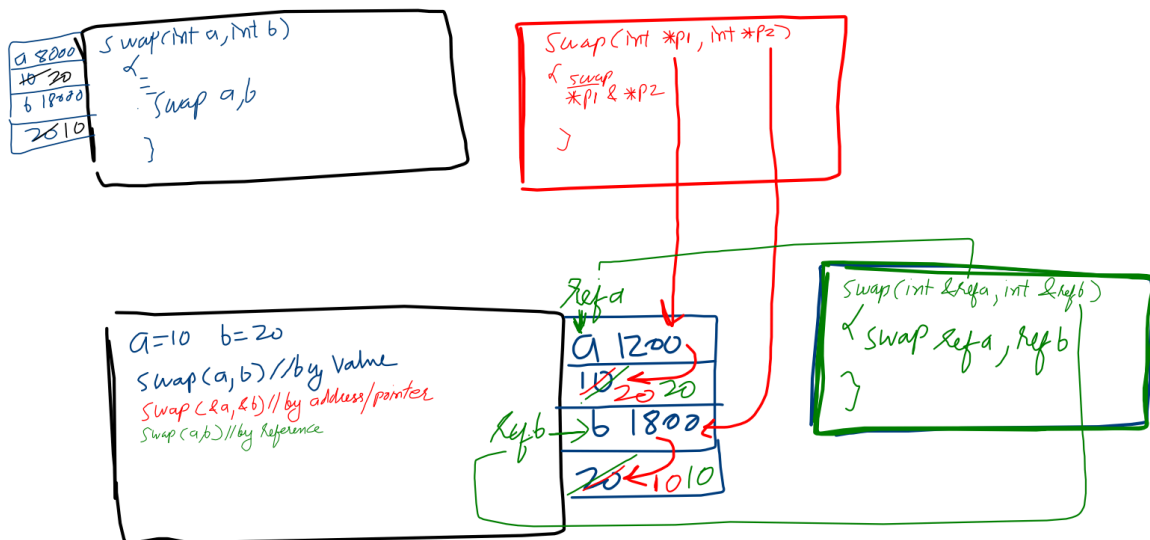
// -----
// PASS BY ADDRESS
// -----
int m = a;
int n = b;

swapByAddress(&m, &n);

cout << "\nAfter Pass by Address:";
cout << "\nm = " << m << ", n = " << n << endl;

return 0;
}

```



Question:

Write a C++ program to create a function that accepts **two parameters: name and nationality**.

- The **name** parameter should be **mandatory**.
- The **nationality** parameter should have a **default value of "Indian"**.
- If the user provides only the name, the function should display the name along with the default nationality.
- If the user provides both name and nationality, the function should display the values provided by the user.

Sample Input 1

Name: Amar

Sample Output 1

Name: Amar

Nationality: Indian

Sample Input 2

Name: John

Nationality: American

Sample Output 2

Name: John

Nationality: American

Problem Definition: GCD of Two Numbers

Write a C++ program to create a function `GCD(no1, no2)` that accepts two integers and returns the **Greatest Common Divisor (GCD)** of the given numbers.

The GCD is the **largest positive number that can divide both given numbers exactly**.

Example

Input:

no1 = 24

no2 = 36

Output:

GCD = 12

Algorithm

1. Start
2. Accept two numbers `no1` and `no2`.
3. Find the remainder when `no1` is divided by `no2`.
4. Replace `no1` with `no2`.
5. Replace `no2` with the remainder.
6. Repeat steps 3–5 until the second number becomes zero.
7. The first number is the GCD.
8. Return the GCD.
9. Stop.

```
int GCD(int no1,int no2)
{
    if(no1%no2==0)
        return no2;
    else
        return
GCD(no2,no1%no2);
}
```

$GCD(18, 12)$
no1 no2

$18 \div 12 = 6 \neq 0$

$GCD(12, 6)$

$GCD(12, 6)$
no1 no2

$12 \div 6 = 0$

$\rightarrow$ no2

```
#include <iostream>
```

```
using namespace std;
```

```
//(condition)? true:false;
```

```
//This is a ternary operator
```

```
//which will evaluate the condition and depending on true or false,
```

```
//the appropriate part will be executed.
```

```
int main()
```

```
{
```

```
    int no1,no2,ans;
```

```
    cout<<"Enter 2 numbers:";
```

```
    cin>>no1>>no2;
```

```
    ans=(no1>no2)?no1:no2;
```

```
    //Using the ternary operator only, find the maximum of three numbers.
```

```
    cout<<"Maximum is:"<<ans;
```

```
    /* auto add=[](int no1,int no2)
```

```
        {
```

```
            return(no1+no2);
```

```
        };
```

```
    cout<<"Addition is:"<<add(11,22);*/
```

```
    return 0;
```

```
}
```

Max of 3

```
#include <iostream>
```

```
using namespace std;
```

```
// (condition) ? true : false;
```

```
// Ternary operator evaluates the condition.
```

```
// If the condition is true, the first value is selected.
```

```
// Otherwise, the second value is selected.
```

```
int main()
```

```
{
```

```
    int no1, no2, no3, ans;
```

```
    cout << "Enter 3 numbers: ";
```

```
    cin >> no1 >> no2 >> no3;
```

```
    ans = (no1 > no2) ?
```

```
        ((no1 > no3) ? no1 : no3) :
```

```
        ((no2 > no3) ? no2 : no3);
```

```
    cout << "Maximum is: " << ans;
```

```
    return 0;
```

```
}
```